Genesis Farm, a commercial farm

We are building Genesis Farm, a commercial farm and estate operating system on top of Taskade Genesis.

Before we build additional infrastructure ourselves, I want you to perform a deep technical audit of the current Taskade platform and your complete available tool stack.

Do not assume that the solution we have described is the best architecture.

I want you to look at the actual capabilities available inside Taskade Genesis and identify whether Taskade already has tools, integrations, browser capabilities, automations, APIs, agent tools, storage mechanisms, authentication features, media handling, notifications or other functionality that could solve these problems more elegantly.

This is not a marketing exercise.

Do not simply tell me Taskade is “scalable”, “powerful” or “flexible”.

Where possible:

- inspect the current Genesis environment
- inspect available automation actions
- inspect agent tools
- inspect integrations
- inspect available JavaScript/browser APIs
- inspect generated application architecture
- inspect available libraries/packages
- inspect API endpoints
- inspect storage mechanisms
- inspect authentication
- inspect deployment capabilities
- inspect application logging and monitoring
- inspect any undocumented or less obvious capabilities accessible within Genesis

Clearly distinguish:

Already supported natively

Can be implemented inside a Genesis application

Requires an automation

Requires an external API

Requires custom development

Not currently possible

Unknown / requires confirmation from Taskade engineering

The business scenario

Genesis Farm will be deployed to farms and estates.

Each customer may receive their own dedicated Taskade Space / Genesis application.

A customer might have:

- 5 employees
- 20 employees
- 50 employees
- 100+ employees

Workers will predominantly use mobile phones in the field.

They may experience:

- excellent 4G/5G
- weak mobile signal
- intermittent signal
- Wi-Fi moving in and out of range
- complete connectivity black spots
- old Android devices
- iPhones
- low battery
- large photographs
- multiple simultaneous users

The application needs to support things including:

personnel location

lone worker

incident reporting

photos

clock in / clock out

tasks

asset locations

estate maps

alerts

management dashboards

WhatsApp notifications

What3Words locations

AI support

diagnostics

automations

1. Solve the rural connectivity problem

This is one of the most important parts of the investigation.

Imagine a worker is standing in a field with poor or no mobile data.

They open Genesis Farm and report:

Fence down

The application captures:

- worker
- date/time
- GPS latitude
- GPS longitude
- GPS accuracy
- incident category
- notes
- photograph

Investigate exactly what Taskade/Genesis can currently do if connectivity disappears during that process.

Can a Genesis application:

- capture GPS while offline?
- capture GPS when mobile data is unavailable?
- access navigator.geolocation?

- retain the GPS result locally?
- capture a photograph while offline?
- retain that photograph locally?
- save a form submission locally?
- use IndexedDB?
- use localStorage?
- use Cache Storage?
- use a Service Worker?
- use Background Sync?
- detect online/offline state?
- detect the connection changing?
- queue unsent submissions?
- retry them automatically?
- retain a queue after the browser/app is closed?
- retain it after the phone reboots?
- prevent duplicate submissions after reconnecting?

If any of these are possible through normal web APIs rather than Taskade-native features, say so.

Most importantly:

Can we implement a reliable store-and-forward outbox directly inside a Genesis application?

Example:

No connectivity

↓

Capture incident locally

↓

Capture GPS locally

↓

Capture photograph locally

↓

Show "Saved on device — not yet sent"

↓

Connection returns

↓

Automatically synchronise with Taskade

↓

Confirm successful server receipt

↓

Remove item from local queue

If yes, design the architecture.

If no, explain exactly what prevents it.

2. Investigate whether Taskade already has an alternative

Do not assume our proposed offline architecture is necessary.

Look across the entire Taskade tool stack for another solution.

Could Taskade already provide something through:

- Projects
- forms
- automations
- webhooks
- media storage
- agents
- mobile functionality
- browser functionality
- PWA behaviour
- external triggers
- notifications
- third-party integrations
- custom HTTP requests
- Taskade APIs
- automation queues

that solves intermittent connectivity?

If there is a better Taskade-native architecture, propose it.

3. What3Words

We are considering integrating What3Words.

The intended model is:

Phone GPS

↓

Latitude / Longitude

↓

Genesis Farm

↓

What3Words API

↓

Three-word location

↓

Store both GPS and W3W

Investigate the best implementation.

Can Genesis:

- call the What3Words REST API directly?
- call it through a Taskade HTTP automation?
- securely store the API key?
- perform calls server-side rather than exposing the key in browser code?
- automatically convert GPS → What3Words?
- convert What3Words → GPS?
- store the result in a Project?
- display the three-word address in the app?
- make it clickable?
- display it on the Leaflet map?
- send it in notifications?
- pass it to an AI agent?

Determine whether there is any existing Taskade integration or mechanism that would make this easier.

Also investigate whether there is any realistic way to perform What3Words conversion without connectivity inside a Genesis PWA.

If native browser offline conversion is not available, say so rather than inventing a solution.

4. WhatsApp

We want incidents and safety alerts to reach the correct people.

Example:

FENCE DOWN

Reported by:

John Smith

Location:

North Field

What3Words:

///word.word.word

GPS:

52.xxxx, 0.xxxx

Accuracy:

±8m

Reported:

14:42

Investigate Taskade's complete WhatsApp capability.

Can Taskade:

- send WhatsApp Business messages?
- trigger messages from automations?
- send messages to different people depending on issue type?

- use templates?
- send images?
- send map links?
- send What3Words links?
- receive WhatsApp replies?
- trigger another automation from a reply?
- allow an AI agent to interact through WhatsApp?
- maintain conversation context?
- escalate to another person if nobody responds?
- detect failed message delivery?
- record delivery status?
- retry failed sends?

Separate outbound notification functionality from two-way conversational functionality.

5. Weak-signal mode

Investigate whether Genesis could intelligently change its behaviour depending on connectivity.

For example:

Strong connection

Send:

- incident
- GPS
- What3Words
- photos
- AI classification
- automations
- notifications

Weak connection

Send only:

- incident type
- worker ID
- coordinates
- timestamp

Then upload photos later.

No connection

Save everything locally and wait.

Can Genesis detect or approximate network quality?

Could it use browser APIs such as navigator.onLine, the Network Information API or another mechanism?

Could Genesis implement an alert-first / media-second architecture?

Design this if possible.

6. Image optimisation

Photos may become one of our biggest field-data problems.

Investigate whether Genesis can:

- resize images on the phone before upload
- compress JPEGs
- convert HEIC
- convert image formats
- remove unnecessary metadata
- create thumbnails
- retain original photos only when necessary
- upload a compressed version first
- upload original media later

Could this happen client-side before the file reaches Taskade?

Could an automation do it?

Could a media tool do it?

Could an agent do it?

Which is cheapest and most reliable?

7. GPS and location intelligence

Inspect everything available around geolocation.

Can Genesis obtain:

- latitude
- longitude
- GPS accuracy

- altitude
- heading
- speed
- timestamp

Can it:

- request updated positions?
- watch location changes?
- stop watching?
- distinguish fresh vs stale location?
- record the last known position?
- calculate distance between two coordinates?
- determine whether somebody is inside a defined estate zone?
- determine whether somebody is within a field polygon?
- trigger an event when somebody enters/leaves an area?

Which of these work while the browser is open?

Which work when it is backgrounded?

Which work when the phone is locked?

This distinction is very important.

Do not imply that browser geolocation provides continuous background personnel tracking unless that is genuinely possible.

8. Map intelligence

Genesis Farm already uses maps.

Investigate everything the current mapping stack can provide that we may not be using.

Look specifically at:

- markers
- clustering
- polygons
- estate boundaries
- geofences
- heat maps
- tracks

- routes
- layers
- live markers
- stale-location indicators
- offline map tiles
- custom map tiles
- field boundaries
- colour-coded personnel
- asset locations
- incident layers
- connectivity black-spot layers

Can we create a connectivity map from synchronisation delays?

Example:

Incident captured:

10:42

Incident reached server:

10:49

Delay:

7 minutes

Could Genesis record these delays and create a heat map showing areas of poor connectivity?

9. Emergency architecture

Lone-worker functionality is safety-sensitive.

Investigate what Taskade can reliably support versus what should remain an external emergency procedure.

Can Genesis:

- produce an emergency alert
- send location
- notify several people

- escalate if unacknowledged
- require acknowledgement
- measure response time
- send repeat alerts
- log the entire event
- trigger WhatsApp
- trigger SMS
- trigger email
- trigger voice/Twilio
- notify another department

Can it detect whether an alert has actually reached the server?

We must distinguish:

Emergency captured locally

from

Emergency transmitted

from

Emergency delivered

from

Emergency acknowledged

Design the states required.

10. Phone-based fall detection / sensors

Investigate whether Genesis can access any useful mobile browser/device capabilities such as:

- accelerometer
- gyroscope
- DeviceMotion
- DeviceOrientation
- vibration
- camera
- microphone
- QR scanner
- barcode scanner

- NFC
- Bluetooth
- push notifications

For each one tell us:

Available reliably in browser/PWA

Available with restrictions

Requires user permission

Requires native mobile app

Not viable

We originally looked at replacing dedicated lone-worker devices with workers' existing phones, so identify anything in Taskade/Genesis that could help.

11. Notifications

Investigate all notification methods available through Taskade:

- in-app
- browser
- web push
- Taskade notifications
- WhatsApp
- SMS
- email
- Microsoft Teams
- Slack
- Telegram
- webhook
- custom API

Which ones can be triggered automatically?

Which provide delivery confirmation?

Which support escalation?

Could Genesis create a standard notification router such as:

Routine

→ In-app

Important

→ In-app + WhatsApp

Critical

→ In-app + WhatsApp + SMS + manager escalation

12. Agents as operational support

Review the Taskade agent tool stack.

We are considering two agents:

Genesis Help

Handles:

- how-to questions
- account questions
- instructions
- FAQs

Genesis Diagnostics

Handles:

- failed automations
- integrations
- broken workflows
- application problems
- failed notifications

Investigate what an agent can actually inspect.

Can it inspect:

- automation run history
- failed automations
- Project records
- agent configuration
- API responses
- webhook responses

- integration status
- application errors
- logs
- media upload failures
- authentication failures

Can it safely:

- retry an automation?
- retry an API call?
- restart a failed workflow?
- repair data?
- modify configuration?
- create a support ticket?
- escalate to a human?

Identify any capabilities we have not considered.

13. Hidden / underused Taskade capabilities

This section is particularly important.

Look across the entire Taskade and Genesis platform and identify capabilities that could improve Genesis Farm that we have not mentioned.

Do not limit yourself to our current design.

Specifically look for:

- underused browser APIs
- less obvious automation triggers
- lesser-known integrations
- agent tools
- AI vision
- document/media processing
- webhooks
- custom HTTP actions
- scheduling
- QR/barcode capability
- Twilio

- WhatsApp
- Telegram
- email triggers
- forms
- structured AI output
- voice/audio
- camera
- push notifications
- mapping libraries
- chart libraries
- local storage
- app analytics
- authentication
- row-level user isolation
- deep links
- custom domains
- usage analytics
- event logs

For each useful discovery explain:

What it is

How Genesis Farm could use it

What problem it removes

Any limitation

14. Concurrency and field load

Analyse the following scenario:

Raynham has:

50 staff

During a busy period:

20 are simultaneously using Genesis

Ten people submit reports within one minute.

Each contains:

- GPS
- text
- 2 images

Each submission causes:

- Project write
- automation
- What3Words lookup
- incident classification
- notification
- WhatsApp message

What bottlenecks exist?

Identify:

- API limits
- automation concurrency
- upload limits
- AI limits
- storage limits
- database/project limits
- webhook limits

If actual numbers are unknown, say so and identify what Taskade engineering needs to answer.

15. Multi-customer architecture

Assume:

Raynham

→ Dedicated Taskade Space

Farm B

→ Dedicated Taskade Space

Farm C

→ Dedicated Taskade Space

Eventually:

100 customers

each potentially having:

20–100 employees.

Analyse whether Taskade's architecture supports this model.

Specifically investigate whether limits apply at:

- app
- Space
- workspace
- user
- account
- organisation
- API
- AI
- automation
- storage

Identify whether one busy customer could affect another.

16. Security

Particularly investigate security around the offline/local architecture.

If Genesis stores:

- employee ID
- location
- photographs
- incident information

temporarily on a phone:

- how is that data stored?
- can it be encrypted?
- can another application read it?
- what happens if the phone is stolen?
- when should cached records be deleted?
- can sensitive records be prevented from remaining locally?

Also investigate how What3Words API credentials and other secrets should be stored.

Never recommend embedding long-lived API secrets directly in public frontend code.

17. Taskade's own recommended architecture

After investigating everything above, forget our proposed architecture temporarily.

Pretend you are the lead Taskade architect responsible for building Genesis Farm.

Design the solution using Taskade's strongest capabilities.

Show:

Mobile worker

↓

Genesis application

↓

location / offline resilience

↓

Taskade Projects

↓

Automations

↓

Agents

↓

What3Words

↓

WhatsApp / notifications

↓

Management dashboard

Change that architecture if Taskade has a better approach.

18. Identify things we are unnecessarily building ourselves

Create a section called:

DO NOT BUILD THIS — TASKADE ALREADY DOES IT

List anything currently in our plans that Taskade already provides.

Then create:

BUILD THIS INSIDE GENESIS

Then:

USE AN EXTERNAL SERVICE

Then:

REQUIRES TASKADE ENGINEERING CONFIRMATION

Then:

CURRENTLY NOT SAFE TO PROMISE CUSTOMERS

This classification is very important.

19. Produce a capability matrix

Finish with a table:

Requirement Taskade native Genesis custom code Automation External API Limitation Confidence

GPS

Offline capture

Local queue

Photo upload

Photo compression

What3Words

WhatsApp

SMS

Lone-worker alert

Geofencing

Push notifications

Offline maps

Background location

AI diagnostics

Automation repair

Multi-customer scale

Use confidence:

Confirmed

Likely

Needs testing

Needs Taskade engineering answer

20. Give us a test plan

Finally, tell us exactly what we should physically test before deploying at Raynham.

Examples:

Test 01: start incident with signal, lose signal during upload.

Test 02: create incident completely offline.

Test 03: close browser before reconnecting.

Test 04: reboot phone before reconnecting.

Test 05: upload ten large iPhone photos.

Test 06: twenty simultaneous users.

Test 07: WhatsApp API unavailable.

Test 08: What3Words API unavailable.

Test 09: Taskade automation fails halfway through.

Test 10: user submits the same incident twice.

Give:

test → expected behaviour → failure behaviour → pass criteria.

And I'd put this sentence right at the end

This is the bit I'd really want Taskade to pay attention to:

The goal of this exercise is not to prove that our current Genesis Farm architecture works. The goal is to discover the best architecture Taskade itself can support. Challenge our assumptions, surface capabilities we have overlooked, identify simpler native solutions, and clearly tell us where Taskade's boundaries genuinely are.